Dubbing Studio: How It Works, and How to Present It

Every number below comes from the code (file names in brackets).

A. The system in one sentence

A YouTube link goes into a 10-stage pipeline (a LangGraph graph). Each stage reads a shared state and saves its files to `cache/<video>/`. The output is the same video with a new soundtrack in the target language and the lips redrawn. Every stage caches its work, so an edit to one line takes seconds instead of minutes.

B. Stage by stage

1. Download [steps/download.py]

- yt-dlp fetches the video, capped at 720p and preferring H.264. The video stream is copied into the output untouched, so higher resolution would only cost download time.
- Two audio files: `audio.wav` (16 kHz mono, analysis only) and `audio_hq.wav` (44.1 kHz stereo, the base of the final mix).
- A clip range can be chosen, or a file uploaded.

2. Separate [steps/separate.py]

- Demucs `htdemucs` splits the audio into vocals and a background bed (music, effects, room tone).
- Vocals give cleaner input for transcription, speakers and pitch. The bed goes under the dub, so the original voice is removed, not just turned down.

3. Transcribe [steps/transcribe.py]

- faster-whisper `large-v3` gives word-level timestamps, with `vad_filter=True` and a 500 ms minimum silence.
- **Silero** is the voice-activity detector bundled inside faster-whisper. It decides where speech is and skips silence, avoiding empty segments and hallucinations.
- Gap filling: a second, more sensitive Silero pass (threshold 0.35) finds speech Whisper skipped (soft voices, voice-over on music). Each gap is transcribed on its own; hallucinations like "thanks for watching" are dropped.

4. Diarize [steps/diarize.py]: who speaks, their gender, how they speak

- **pyannote** speaker-diarization-community-1 gives speaker turns and handles overlap (needs HF_TOKEN). Fallback: **ECAPA** voiceprints (SpeechBrain) on 1.5 s windows hopping 0.75 s, clustered.
- Second opinion: each speaker's windows are re-clustered by voiceprint. A speaker is split only if two clearly different voices appear (similarity below 0.35, each at least 1.5 s).
- Words are split at speaker changes, then joined into sentences, never across two speakers.
- Faces [video/faces.py]: ArcFace identities (insightface) plus mouth movement can split

characters the voice model merged. Only acts when more talking faces than speakers are seen.
- Merge: two "speakers" with voice similarity of at least 0.50 are merged (e.g. a raised voice).
- Gender: two wav2vec2 classifiers averaged over all of a speaker's lines. Faces overrule them when clear. Pitch (180 Hz or higher = female) is only the fallback.
- Delivery [speech/prosody.py]: pitch, range, loudness, rate and voicing of each line are scored against that speaker's own baseline (robust z-score), giving labels like calm, energetic, quick.
- Indic sources: Bodhan indic-transcribe rewrites each line's text; Whisper still supplies timing.

5. Refine [steps/refine.py]

- An LLM (Claude Sonnet via OpenRouter) sees Whisper's and Bodhan's text, neighbouring lines and the title, and writes what was actually said, without translating.
- It marks lines high/low confidence and speech/singing/noise (noise is not dubbed). Chunks of 40 lines.

6. Translate [steps/translate.py]

- **Indic** (default): Bodhan indic-translate line by line, then the **dubbing director** [text/director.py], one LLM pass over the whole conversation, fixes gender agreement in verbs, spoken register, forms of address and timing, and splits lines holding two speakers.
- **Concise**: Bodhan's lines kept, only shortened to fit; no director. **LLM only**: the LLM translates the whole dialogue at once.
- Time budget [text/length_fit.py]: characters allowed = slot seconds x characters per second x speaker tempo x 0.95. The slot is the line's length plus at most 0.3 s of borrowed silence.
- Characters per second is measured per voice [voice/tts_rate.py, stored in cache/_tts_rates.json; default 14], because Tamil packs more characters per syllable than Hindi.
- Tempo [speech/pace.py] = the speaker's syllables per second compared with 5.0, clamped to 0.75 to 1.3.
- A line more than 1.15x over budget is rewritten shorter. A per-video glossary of names and words goes to every translator.

7. Synthesize [steps/synthesize.py]

- Casting [voice/casting.py]: one voice per speaker for the whole video. The most-talking speaker gets the native voice; others get the unused voice with the closest measured pitch. UI picks always win.
- Engines: Bodhan indic-speak (about 45 voices, 22 languages), Sarvam Bulbul (11 languages), and in International mode Kokoro and Piper. Lines run in parallel through a rate-limit gate [bodhan_gate.py] that reads Bodhan's own limits from response headers and obeys 429 retry-after.
- Style: one pace style per speaker (lecture, audiobook, news...) chosen from their tempo.
- **Refit** [voice/refit.py], at most 2 rounds: every real clip is measured. A clip needing more than 1.15x (1.30x if the slot is under 1.5 s) is rewritten shorter and re-spoken. A dub under 80% of the original (lines of 1.5 s or more) is lengthened so lips don't move in silence.

8. Align [steps/align.py]: the speed question

- Slot = time from the line's start until 0.08 s before the next line, borrowing at most 1.0 s of

following silence. Two dubs never overlap.

- need = clip length / slot.
  - need 1.0 or less and clip under 95% of the original: slow gently; **floor 0.90x**.
  - need 1.0 or less otherwise: leave as is.
  - need above 1.0: **sped up by exactly what it needs. No hard ceiling; words are never cut.**
- Up to **1.10x** is unnoticeable. Above **1.25x** the line is still sped up but logged as audible (e.g. "line 10 needs 1.58x").
- Smoothing: neighbouring lines of one speaker closer than 1.0 s stay within 0.15 speed of each other (the slower line is sped up, never the fast one slowed).
- Stretching uses **rubberband** (pitch-preserving); fallback ffmpeg `atempo`. 15 ms fades on every clip; silence trimmed at -40 dB.
- **Voice cloning (Seed-VC)** also runs here: each TTS line is converted to the original speaker's timbre using up to 25 s of their clean lines as reference. A clone is rejected if its length differs by more than about 2%, its loudness envelope correlation is under 0.6, its pitch is more than 5 semitones off, or its gender is wrong. Rejected lines retry with the speaker's shared reference, then fall back to plain TTS.

9. Lip sync [steps/lipsync.py]

- Smart planner [video/smart_sync.py]: each line is cut into shots at camera cuts; every face turned no more than about 40 degrees is cropped and scored with **SyncNet** against the original voice. A speaking face matches at an offset of 4 frames or less with confidence of at least 1.5 (0.8 for a lone face in a shot over 3 s). Listeners and photos score badly and are not synced: "when unsure, don't sync".
- **LatentSync 1.6** (diffusion) redraws the mouth. About 10 GB GPU per worker; 2 workers on 24 GB; renders at 25 fps.
- **RAFT optical flow** [video/frame_interp.py] converts 25 fps to the video's frame rate; fallback ffmpeg blending.
- Without enough free GPU memory lip sync is skipped; the dub still works. Lip sync changes video only, never audio.

10. Assemble [steps/assemble.py]

- Outside dialogue the original audio stays; inside dialogue the Demucs bed is crossfaded in (10 ms).
- Each dub is placed at its time. [audio/mixing.py] keeps it at least 8 dB above the music, lifting the dub by at most 6 dB and ducking the music by at most 8 dB.
- Loudness normalised to **-14 LUFS**, true peak ceiling -1 dBTP, saved as AAC. Lip-synced shots are overlaid onto the original frames only for their own time spans, so timing cannot drift.

Extras

- **Song mode**: sung lines are detected by held notes (sung chant 0.35, speech at most 0.06) and the AudioSet AST classifier. By default the original singing is kept; optionally Lyria 3 re-sings the lyrics (about $0.04 per phrase, max 12).
- **Ask / edit tab** [edits/edit_planner.py]: "make line 5 sadder" becomes typed, validated actions (about 15 kinds); only the affected stages re-run.
- Version history (restorable), audition of one line before applying, ETA learned from past runs.

C. Code structure

```
Dubbing_Studio/
  scripts/setup_gpu.sh        one-shot installer
  .env                        API keys
  dubbing/
    server/   FastAPI: app.py (API), jobs.py (one-at-a-time queue + SSE),
              runner.py, projects.py (no DB: a project is a cache folder),
              revise.py / audition.py / versions.py / eta.py
    pipeline/ graph.py (4 LangGraph graphs), state.py (Segment, PipelineState)
      steps/  the 10 stages
      speech/ speaker_turns, gender, prosody, pace, bodhan_asr
      text/   director, dialogue, length_fit, line_by_line, glossary, llm
      voice/  casting, tts, catalogue, voice_clone, refit, styles, tts_rate
      audio/  mixing     video/ faces, smart_sync, shot_planner, compose, frame_interp
      music/  song_detect, song_mode     edits/ edit_planner
    web/      Vue 3 (no build step) UI, d3 globe
    LatentSync/ SeedVC/ Separator/  third-party models in their own venvs
    cache/    outputs per video     tests/ tools/
```

Four design ideas: (1) everything cached by input hash; (2) four graphs (full, edit, re-voice, lip-sync-later) reuse the same stage functions; (3) fail soft: every model has a fallback (pyannote to ECAPA, rubberband to atempo, clone to TTS, lip sync skipped without a GPU); (4) words are never cut: the line is rewritten shorter, then sped up only as far as needed.

D. Video 1: Project demonstration (about 8-10 min, at least 3 diverse cases)

For each case: show the original, then the dub, then one thing the log proves.

1. **Two-person small talk to Hindi or Tamil**: speaker detection, gender-correct verbs, one consistent voice per person.
2. **Single-speaker lecture or talk to another language (Telugu, Bengali)**: pace, timing and lip sync; open the log to show "Fitted N clips ... worst need X".
3. **Hard content: film clip or trailer with music, many characters or a song**: face-based speaker splits, music kept, song detection.
4. **Bonus: Ask tab**: "make line 5 more casual", only that line redoes in seconds, then restore the old version from history.

Run each clip once in advance; rate-limit waits (60 s each) are not worth showing live.

E. Video 2: Code walkthrough (about 10-12 min)

1. The problem in 30 s, then the 10-stage table from the README.
2. Setup: `bash scripts/setup_gpu.sh`, the `.env` keys (say what each is for, show no values), `python -m server`, open localhost:8000.
3. The folder tree from section C.
4. graph.py and state.py: the graphs, and the Segment object every stage fills in.
5. Three deep dives: diarize (pyannote, classifiers, faces); the length-fitting loop (translate budget, refit, align); lip sync (SyncNet then LatentSync).

6. Robustness: the Bodhan gate, caching, fallbacks, tests.
7. Close with the Ask tab as the feature that sets this apart.

If asked: the default is an LLM director plus Bodhan, so wording quality depends on those services. Lines needing more than 1.25x are sped up and flagged in the log, never cut.